

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (4) **Computer Structure**

Bachelor in Computer Science and Engineering
Bachelor in Applied Mathematics and Computing
Dual Bachelor in Computer Science and Engineering and Business Administration



Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Instruction formats, addressing modes, and instruction sets
4. Floating point in assembly
5. Procedure calls and stack convention

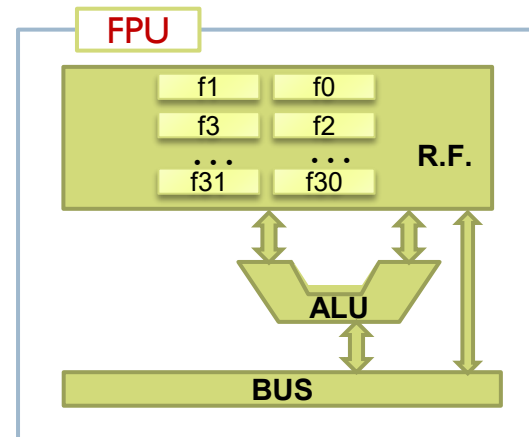
Register File (floating-point)

ABI Name	Name	Usage
ft0...ft7	f0 ... f7	Temporals (like t...)
fs0...fs1	f8 ... f9	There are saved (like s...)
fa0...fa1	f10 ... f11	Arguments/return (like a...)
fa2...fa7	f12 ... f17	Arguments (like a...)
fs2...fs11	f18 ... f27	There are saved (like s...)
ft8...ft11	f28 ... f31	Temporals (like t...)

- ▶ There are 32 registers
- ▶ The ft0 register does not have its value set to 0.
- ▶ In the single precision extension, the registers are 32 bits (4 bytes).
- ▶ In the double precision extension, the registers are 64 bits (8 bytes) and can store:
 - ▶ Single precision values in the lower 32 bits of the register
 - ▶ Double precision values in the 64 bits of the register

Floating-point register file

- ▶ There are 32 floating-point registers in addition to the integer registers:
 - ▶ From f0 to f31
- ▶ Copy registers (.s .d):
 - ▶ `fmv.s rd rs` # `rd = rs`
- ▶ Common arithmetic operations (.s .d):
 - ▶ `fadd.s rd rs1 rs2` # `rd = rs1 + rs2`
 - ▶ `fsub.s rd rs1 rs2` # `rd = rs1 - rs2`
 - ▶ `fmul.s rd rs1 rs2` # `rd = rs1 * rs2`
 - ▶ `fdiv.s rd rs1 rs2` # `rd = rs1 / rs2`
 - ▶ `fmin.s rd rs1 rs2` # `rd = min(rs1, rs2)`
 - ▶ `fmax.s rd rs1 rs2` # `rd = max(rs1, rs2)`
 - ▶ `fsqrt.s rd rs1` # `rd = sqrt(rs1)`
 - ▶ `fmadd.s rd rs1 rs2 rs3` # `rd = rs1 x rs2 + rs3`
 - ▶ `fmsub.s rd rs1 rs2 rs3` # `rd = rs1 x rs2 - rs3`
 - ▶ `fabs.s rd rs` # `rd = |rs|`
 - ▶ `fneg.s rd rs` # `rd = -rs`

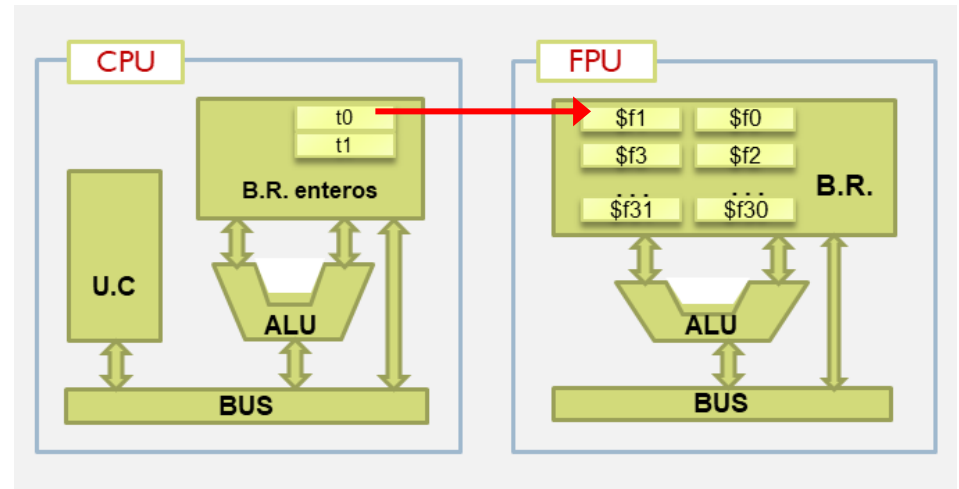


Copy instructions

(integer registers $\leftrightarrow$ floating-point registers)

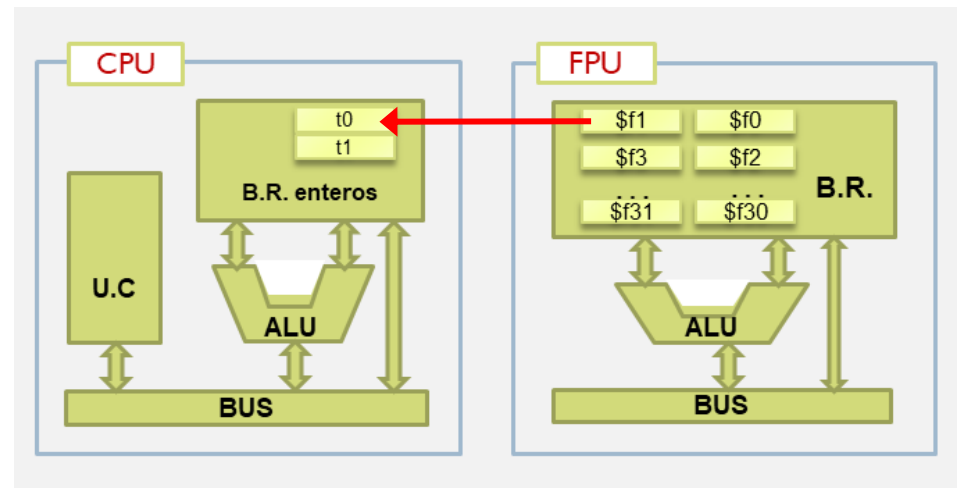
`fmv.w.x rd rs`

- Copy from integer register `rs` to a floating register `rd` (*single precision*)



`fmv.x.w rd rs`

- Copy from floating register `rs` (*single precision*) to a integer register `rd`



Conversion operations (1/3)

integer $\leftrightarrow$ simple precision

► `fcvt.w.s rd, rs`

- Convert from simple precision (value in floating register `rs`) to a 32-bit integer 32 **with** sign (integer register `rd`).

► `fcvt.wu.s rd, rs`

- Convert from simple precision (value in floating register `rs`) to a 32-bit integer 32 **withou** sign (integer register `rd`).

► `fcvt.s.w rd, rs`

- Convert a 32-bit integer 32 **with** sign (value in integer register `rs`) to a simple precision (floating register `rd`).

► `fcvt.s.wu rd, rs`

- Convert a 32-bit integer 32 **without** sign (value in integer register `rs`) to a simple precision (floating register `rd`).

Conversion operations (2/3)

integer $\leftrightarrow$ double precision

► `fcvt.w.d rd, rs`

- Convert from double precision (value in floating register `rs`) to a 32-bit integer 32 **with** sign (integer register `rd`).

► `fcvt.wu.d rd, rs`

- Convert from double precision (value in floating register `rs`) to a 32-bit integer 32 **withou** sign (integer register `rd`).

► `fcvt.d.w rd, rs`

- Convert a 32-bit integer 32 **with** sign (value in integer register `rs`) to a double precision (floating register `rd`).

► `fcvt.d.wu rd, rs1`

- Convert a 32-bit integer 32 **without** sign (value in integer register `rs`) to a double precision (floating register `rd`).

Conversion operations (3/3)

double precision <-> simple precision

▶ `fcvt.s.d rd, rs1`

- ▶ Convert from double precision (value in floating register rs) to simple precision (floating register rd).

▶ `fcvt.d.s rd, rs`

- ▶ Convert from simple precision (value in floating register rs) to double precision (floating register rd).

Classification of floating-point numbers

- ▶ `fclass.s rd, rs1` (simple precision)
- ▶ `fclass.d rd, rs1` (double precision)
- ▶ Save in `rd` the type of the floating-point number of the register `rs1`:

Value in rd	Meaning
0	-Inf
1	Negative normalized number
2	Negative non-normalized number
3	-0
4	+0
5	Positive non-normalized number
6	Positive normalized number
7	+Inf
8	NaN (non-quiet)
9	NaN (quiet)

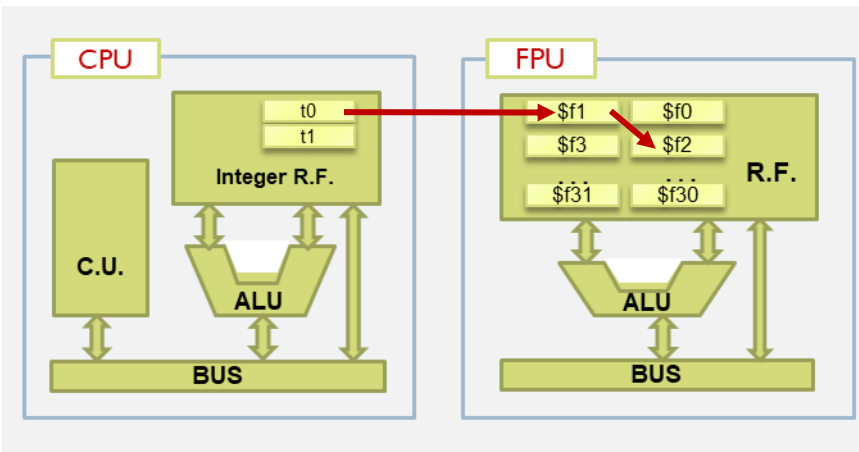
Example

```
float PI    = 3,1415;
int  radio = 4;
float length;

length = PI * radio;
```

```
.text
main:
```

```
li t0, 0x40490E56
    # does not exist li.s
    # 0x40490E56 is the
    # representation IEEE754
    # in hexadecimal for 3.1415
fmv.w.x ft0, t0    # ft0 ← t0
li      t1  4      # 4 in Ca2
fcvt.s.w ft1, t1    # 4 in ieee754
fmul.s  ft0, ft0, ft1
```



Comparison instructions (floating-point registers)

► Simple precision:

- `feq.s rd, rs1, rs2` if ($rs1 == rs2$) $rd = 1$; else $rd = 0$
- `fle.s rd, rs1, rs2` if ($rs1 \leq rs2$) $rd = 1$; else $rd = 0$
- `flt.s rd, rs1, rs2` if ($rs1 < rs2$) $rd = 1$; else $rd = 0$

► Double precision:

- `feq.d rd, rs1, rs2` if ($rs1 == rs2$) $rd = 1$; else $rd = 0$
- `fle.d rd, rs1, rs2` if ($rs1 \leq rs2$) $rd = 1$; else $rd = 0$
- `flt.d rd, rs1, rs2` if ($rs1 < rs2$) $rd = 1$; else $rd = 0$

Definition of static data (at compile time)

label (address)

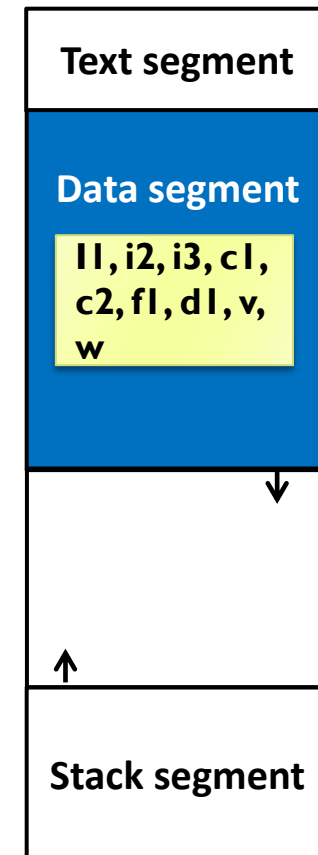
Data type (directive)

value

```
.data
cadena: .string "Hello world\n"
i1: .word 10      # int    i1=10
i2: .word -5      # int    i2=-5
i3: .half 300     # short  i3=300
c1: .byte 100     # char   c1=100
c2: .byte 'a'     # char   c2='a'
f1: .float 1.3e-4 # float  f1=1.3e-4
d1: .double .001  # double d1=0.001

# int v[3]={0,-1,0xffffffff}; int w[100];
v: .word 0, -1, 0xffffffff
w: .zero 400
```

0x00



.text

.data

Representation of basic data types (1/3)

```
// boolean
bool_t b1 ;
bool_t b2 = false ;
```

```
// character
char c1 ;
char c2 = 'x' ;
```

```
// integers
int res1 ;
int op1 = -10 ;
```

```
// floating point
float f0 ;
float f1 = 1.2 ;
double d2 = 3.0e10 ;
```

.data

```
# boolean
b1:  .zero 1          # 1 bytes
b2:  .byte 0          # 1 byte
```

```
# character
c1:  .byte          # 1 bytes
c2:  .byte 'x'      # 1 bytes
```

```
# integers
res1: .zero 4        # 4 bytes
op1:  .word -10      # 4 bytes
```

```
# floating point
f0:  .float          # 4 bytes
f1:  .float 1.2       # 4 bytes
d2:  .double 3.0e10   # 8 bytes
```

Memory operations: floating point

Memory access (floating point)		
<code>flw rs, 10(rs1)</code>	<code>rs = Memory[rs1+10]</code>	Loads the single precision value stored at address (rs1+10) into the floating-point register rd.
<code>fsw rs, 10(rs1)</code>	<code>Memory[rs1+10] = rs</code>	Stores the single value precision of the rs register at address (rd1+10).
<code>fld rd, 10(rs1)</code>	<code>rd = Memory[rs1+10]</code>	Loads the double precision value stored at address (rs1+10) into the rd register.
<code>fsd rd, 10(rs1)</code>	<code>Memory[rs1+10] = rd</code>	Stores the double precision value of the rs register at address (rd1+10).

Elemental data types

Float

```
float  result ;
float  op1 = 100 ;
float  op2 = 2.5
...
```

```
main ()
{
    result = op1 + op2 ;
    ...
}
```

.data

.align 2

```
    resultado:  .zero 4 # 4 bytes
    op1:        .float 100
    op2:        .float 2.5
```

...

.text

```
main: flw      ft0, op1(x0)
      flw      ft1, op2(x0)
      fadd.s   ft3, ft1, ft2
      fsw      ft3, result(x0)
      ...
```

Elemental data types

Double

```
double resultado ;  
double op1 = 100 ;  
double op2 = -10.27 ;  
...
```

```
main ()  
{  
    resultado = op1 + op2 ;  
    ...  
}
```

.data

.align 3

```
resultado:  .zero 8  
op1:        .double 100  
op2:        .double -10.27
```

...

.text

```
main: fld      ft0 op1(x0)  
      fld      ft1 op2(x0)  
      fadd.d    ft3 ft1 ft2  
      fsd      ft3 resultado(x0)  
      ...
```


Branches with floating-point numbers

Jump to the label if
ft1 < ft2

```
flt t0, ft1, ft2
bne t0, x0, label1
. . .
label1:
```

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (4) **Computer Structure**

Bachelor in Computer Science and Engineering
Bachelor in Applied Mathematics and Computing
Dual Bachelor in Computer Science and Engineering and Business Administration



Example. Calculate

$$\sum_{i=1}^N \frac{1}{2^i}$$

```
.data

N:          .word  10           # Upper limit
one_f:      .float 1.0         # 1.0 in floating-point format
two_f:      .float 2.0         # 2.0 in floating-point format

.text

main:
    la      t0, N
    lw      t0, 0(t0)          # t0 = N = 10

    la      t1, one_f
    flw     ft0, 0(t1)         # ft0 = 1.0 (numerator)

    la      t1, two_f
    flw     ft1, 0(t1)         # ft1 = 2.0 (base of the power)

    fmv.w.x fa0, zero          # accumulates the result

    # Inicializar variables del bucle
    li      t1, 1              # t1 = i = 1
    fmv.s   ft2, ft1           # ft2 = 2.0 (will represent 2^i, starting at 2^1)

    # to be continued...
```

Example. Calculate

$$\sum_{i=1}^N \frac{1}{2^i}$$

```
loop:

    bgt t1, t0, end_loop

    # Calculate current term: ft3 = 1.0 / (2^i)
    fdiv.s ft3, ft0, ft2 # ft3 = ft0 / ft2

    # Add to the total accumulator: fa0 = fa0 + ft3
    fadd.s fa0, fa0, ft3

    # Update for the next iteration
    fmul.s ft2, ft2, ft1 # Multiplies the power by 2 for the next power (2^(i+1))
    addi t1, t1, 1 # i = i + 1

j loop

end_loop:

# Print the result
li a7, 2
ecall

li a7, 10    # Syscall 10 = exit
ecall
```